

4A__AbstractAnalysis

SJTU AI4Math 2026

目录

1 第一章 极限的逐层抽象：从实数到度量空间与拓扑空间	4
1.1 epsilon-delta 语言	4
1.1.1 为什么离开 $\mathbb{R}$	4
1.2 度量空间：从距离到开球	4
1.3 拓扑空间：忘掉距离，只保留“什么算附近”	5
1.4 本章收束	7
2 第二章 Filter: Mathlib 中极限的统一语言	8
2.1 先从数学定义开始	8
2.1.1 三条公理	8
2.1.2 空集	8
2.1.3 Lean	8
2.1.4 滤子记录的是“尺度”，不是轨迹	9
2.2 最基本的构造：principal 与 pure	9
2.2.1 主滤子 principal	9
2.2.2 纯滤子 pure	9
2.3 滤子的序、顶底元与格运算	10
2.3.1 序	10
2.3.2 顶元与底元	10
2.3.3 交 $F \sqcap G$ 与并 $F \sqcup G$	11
2.3.4 完备格与条件完备格	11
2.4 NeBot: 何时排除退化	12
2.5 邻域滤子与集合内邻域	12
2.5.1 普通、集合内与去心邻域	12
2.6 无穷方向：atTop 与 atBot	13
2.6.1 再比较扩充数与 atTop	13
2.7 Eventually: 滤子的命题语言	14
2.7.1 定义与基本推理	14
2.8 Frequently: 无法被最终排除	15
2.9 map 与 comap: 沿函数搬运滤子	16

2.9.1	map: 推送源端行为	16
2.9.2	comap: 拉回目标条件	16
2.9.3	map-comap 伴随与复合	17
2.10	EventuallyEq: 最终相等与局部数学	17
2.10.1	严格定义	17
2.10.2	在邻域滤子下: 局部相等	18
2.10.3	函数芽 germ	18
2.11	EventuallyLE 与渐近分析	18
2.11.1	最终不等式	18
2.11.2	Big-O 把滤子作为趋近参数	19
2.12	Tendsto: 统一极限的一个关系	19
2.12.1	严格定义	19
2.12.2	三个最小例子	20
2.12.3	为什么 Mathlib 选择 Filter	20
3	第三章 极限与连续性的形式化	21
3.1	极限词典: 只替换滤子参数	21
3.2	epsilon-delta 与 Tendsto 的等价	21
3.2.1	目标为度量邻域时先展开 epsilon	21
3.2.2	源也为度量邻域时再展开 delta	22
3.3	点极限、无穷远与数列极限	23
3.3.1	普通点与去心点	23
3.3.2	数列只是定义域为自然数的函数	23
3.3.3	源趋于无穷与值趋于无穷	23
3.4	左极限、右极限与两侧合并	24
3.5	连续性: ContinuousAt 及其组合	24
3.5.1	点态连续的定义	24
3.5.2	连续映射复合	25
3.6	极限证明的三种层次	25
3.7	唯一性为何需要结构条件	26
4	第四章 微分: 从极限到 Fréchet 导数	27
4.1	导数是局部线性逼近	27
4.2	为什么以 Fréchet 导数为基础	27
4.3	HasDerivAt: 一元导数的接口	28
4.4	链式法则是连续线性映射的复合	28
4.5	常见函数求导: 由原子定理组合	29
5	第五章 积分、收敛与分析形式化的特点	31
5.1	Measure: 集合大小的类型化对象	31
5.2	为什么以 Lebesgue 积分而非 Riemann 积分为基础	31
5.3	MeasureTheory.integral 与 Integrable	32

5.4	几乎处处: forall-m 是一个 Filter 量词	32
5.5	控制收敛定理如何表达	33
5.6	全篇收束: 分析形式化的三个特点	34
5.6.1	高度抽象, 但抽象有明确职责	34
5.6.2	大量 typeclass 用于传递结构	34
5.6.3	存在性与条件管理比代数更复杂	34

1 第一章 极限的逐层抽象：从实数到度量空间与拓扑空间

本章沿着一条链推进：

$$|x - a| < \delta \longrightarrow d(x, a) < \delta \longrightarrow x \in B(a, \delta) \longrightarrow x \in U \in \mathcal{N}(a).$$

1.1 ε - δ ：实数定义

设 $f : \mathbb{R} \rightarrow \mathbb{R}$ ，并给定 $a, b \in \mathbb{R}$ 。去心极限

$$\lim_{x \rightarrow a} f(x) = b$$

意味着

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x, \quad 0 < |x - a| < \delta \implies |f(x) - b| < \varepsilon. \quad (1)$$

连续性是在极限中取 $b = f(a)$ ，并允许 $x = a$ ：

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x, \quad |x - a| < \delta \implies |f(x) - f(a)| < \varepsilon. \quad (2)$$

目标先指定精度 ε ，源端再给出可依赖于它的半径 δ 。但两个绝对值都只是在回答同一个问题：两个点有多近？

第一次抽象：寻找最小依赖

极限定义的核心不是实数的乘法，而是输入距离 $|x - a|$ 与输出距离 $|f(x) - b|$ 。把它们替换为

$$|x - a| \rightsquigarrow d_X(x, a), \quad |f(x) - b| \rightsquigarrow d_Y(f(x), b)$$

便得到度量空间之间的同一陈述。逐行检查证明实际使用的运算，并把恰好提供这些运算的结构写进假设，这就是形式化中的“找最小依赖结构”。

1.1.1 为什么离开 $\mathbb{R}$

抽象的目标不是最大的一般性，而是恰好保留证明所需的结构。若只用距离，就假设 `[MetricSpace X]`；若只用开集和邻域，就继续降到拓扑结构。这样，函数空间中的函数可以成为“点”，同一个复合定理只需证明一次，而定理所需的假设也会在类型中被明确记录。

1.2 度量空间：从距离到开球

度量空间给类型 X 配备距离 $d_X : X \times X \rightarrow \mathbb{R}$ ，满足非负性、对称性、三角不等式以及 $d_X(x, y) = 0 \leftrightarrow x = y$ 。连续性的 ε - δ 形式只是把 (2) 中的绝对值替换为距离：

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x, \quad d_X(x, a) < \delta \implies d_Y(f(x), f(a)) < \varepsilon. \quad (3)$$

去心极限的对应式只需保留 $x \neq a$ 并把目标点换成 b ：

$$0 < d_X(x, a) < \delta \implies d_Y(f(x), b) < \varepsilon.$$

说明定义并不依赖 $\mathbb{R}$ 的具体身份。

Lean 用 `typeclass` 表示：

Lean 中只要求距离结构

```
variable X Y : Type*
variable [MetricSpace X] [MetricSpace Y]

#check dist
#check Metric.ball

example (x y : X) (r : ℝ) :
  y ∈ Metric.ball x r ↔ dist y x < r := by
  simp
```

`[MetricSpace X]` 表示 X 不是特指 $\mathbb{R}$ ，但 Lean 已知它带有合法的度量结构。同一个 `dist x y` 因而可以作用于实数、向量或函数。`Metric.ball` 甚至只需要更弱的 `PseudoMetricSpace`；这里不展开两者差别，只记住：一个定义需要什么，库就尽量只要求什么。

定义开球

$$B_X(a, \delta) = \{x \in X : d_X(x, a) < \delta\}.$$

于是

$$d_X(x, a) < \delta \iff x \in B_X(a, \delta),$$

而局部估计可写成

$$B_X(a, \delta) \subseteq f^{-1}(B_Y(b, \varepsilon)).$$

这一步把数值比较改写为集合包含关系，是进入拓扑的真正入口：

$$\boxed{\text{数值比较} \longrightarrow \text{集合包含关系}}.$$

例 1.2：同一度量定义作用于函数空间

在连续函数空间上可用一致距离

$$d_\infty(u, v) = \sup_{t \in [0, 1]} |u(t) - v(t)|.$$

此时 $d_\infty(u_n, u) \rightarrow 0$ 表示一致收敛，而不是逐点收敛。公式的外形没有改变，改变的只是点的类型和它所带的距离。

1.3 拓扑空间：忘掉距离，只保留“什么算附近”

极限通常不关心 $d(x, a)$ 的具体数值，只关心 x 能否进入 a 的任意小范围。度量给出这些小范围的具体模型——开球；第二次抽象则把半径和距离数值全部忘掉，只保留哪些集合算作附近。

拓扑空间是在类型 X 上指定一族开集 $\mathcal{T} \subseteq \mathcal{P}(X)$ ；这族集合包含 $\emptyset, X$ ，对任意并和有限交封闭。集合 $V \subseteq X$ 是 a 的邻域，若存在开集 U 使

$$a \in U \subseteq V.$$

邻域本身不必开；所有邻域共同组成 $\mathcal{N}(a)$ 。由度量产生的拓扑以开球为局部基，但一般拓扑不再记住任何距离数值。

第二次抽象：定量结构变成定性结构

$$\begin{array}{ccccc} \mathbb{R} & \implies & \text{MetricSpace} & \implies & \text{TopologicalSpace} \\ |x - y| & & d(x, y) & & \text{开集与邻域} \\ \text{定量} & & \text{定量} & & \text{定性} \end{array}$$

每向右一步都忘掉一些数据，但保留当前问题所需的结构。只讨论连续性和极限时，邻域原像往往已经足够；只有收敛速度或 Lipschitz 常数等问题才需要距离数值。

在 Lean 中，度量空间会给出一个拓扑实例；只谈开集和邻域的定理则只需 `[TopologicalSpace X]`：

Lean 中的拓扑接口

```
variable X Y Z : Type*
variable [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z]
variable f : X → Y g : Y → Z x : X

#check TopologicalSpace
#check IsOpen
#check nhds
#check ContinuousAt
#check ContinuousAt.comp

example (hg : ContinuousAt g (f x)) (hf : ContinuousAt f x) :
  ContinuousAt (g ∘ f) x :=
  hg.comp hf
```

`nhds x` 是 Mathlib 对 $\mathcal{N}(x)$ 的接口，邻域语言中的连续性是

$$V \in \mathcal{N}(f(a)) \implies f^{-1}(V) \in \mathcal{N}(a). \quad (4)$$

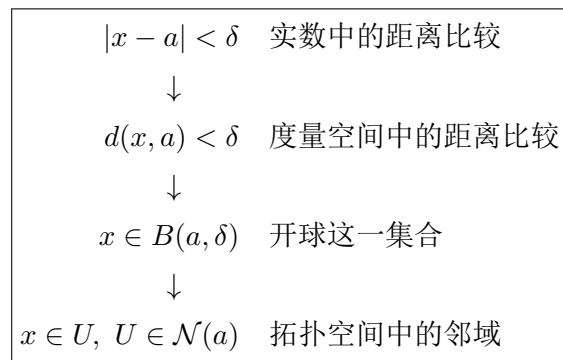
例 1.3：连续复合只需原像的复合

若 f 在 a 连续、 g 在 $f(a)$ 连续，对任意 $g(f(a))$ 的邻域 W ，先有 $g^{-1}(W)$ 是 $f(a)$ 的邻域，再有

$$f^{-1}(g^{-1}(W)) = (g \circ f)^{-1}(W)$$

是 a 的邻域。因此 $g \circ f$ 连续。证明没有使用距离，也没有重新选择 ε 和 δ 。

1.4 本章收束



实数极限中的 $|x - a|$ 不是本质，趋近于 a 才是本质。形式化的工作，是把证明实际使用的结构逐层写进类型，而不是把具体公式机械替换成新字母。到这里我们已经有了邻域系统；下一章再回答：为什么 Mathlib 不用普通的集合族，而用 Filter 来记录它？

2 第二章 Filter: Mathlib 中极限的统一语言

滤子不是专为 Lean 发明的编码技巧。它首先是一个普通的数学对象：在固定类型 α 上，挑出一族被视为“足够大”或“最终区域”的集合。点附近、指标充分大、集合内以及几乎处处，都可以用同一套公理表达。

2.1 Filter(α)

2.1.1 三条公理

固定类型 α 。滤子是幂集 $\mathcal{P}(\alpha)$ 的一个子集

$$\mathcal{F} \subseteq \mathcal{P}(\alpha),$$

其中 $A \in \mathcal{F}$ 读作“ A 是一个最终区域”。它满足：

1. $\alpha \in \mathcal{F}$;
2. 若 $A \in \mathcal{F}$ 且 $A \subseteq B \subseteq \alpha$, 则 $B \in \mathcal{F}$;
3. 若 $A, B \in \mathcal{F}$, 则 $A \cap B \in \mathcal{F}$ 。

第一条说恒真条件总是可接受；第二条说放宽要求不会失效；第三条说有限多个最终条件可以同时成立。滤子记录哪些集合足以描述我们关心的“后面”或“附近”。

滤子的数学类型

$$\mathcal{F} \subseteq \mathcal{P}(\alpha), \quad A \in \mathcal{F} \quad (A \subseteq \alpha).$$

例 2.1: 自然数的尾集

令 $\mathcal{F}$ 收集所有包含某个尾集

$$\{n \in \mathbb{N} : N \leq n\}$$

的集合。全体 $\mathbb{N}$ 属于 $\mathcal{F}$ ；包含尾集的集合向上封闭；两个尾集的交包含从 $\max(N_1, N_2)$ 开始的尾集。因此它满足三条公理，正是“对充分大的自然数成立”的数学对象。

2.1.2 空集

三条公理允许 $\emptyset \in \mathcal{F}$ 。这时每个集合都属于 $\mathcal{F}$ ，称为退化滤子。先允许它，可以让所有滤子自然组成完备格，任意交、并或限制操作都有结果；需要实际的非退化趋近时，再单独加入条件 $\mathcal{F} \neq \perp$ 。这个设计把“构造总有定义”和“结论需要存在性”分开，稍后会用 Mathlib 的 `F.NeBot` 精确表达。

2.1.3 Lean

现在才对应 Mathlib 的声明：

$$F : \text{Filter}(\alpha), \quad A : \text{Set}(\alpha), \quad A \in F : \text{Prop}.$$

Mathlib 的类型入口

```
#check Filter          -- Type u → Type u
#check Filter.sets      -- Filter α → Set (Set α)
#check Filter.univ_sets
#check Filter.sets_of_superset
#check Filter.inter_sets
```

`Filter.sets` 只是上面数学定义中的 $\mathcal{F}$ ；字段名不会改变对象的含义。

2.1.4 滤子记录的是“尺度”，不是轨迹

数列是一条具体轨迹 $u : \mathbb{N} \rightarrow X$ ；而邻域滤子 $\mathcal{N}(a)$ 不规定一条进入 a 的路径，只记录每个足够靠近 a 的区域。这种分离使同一个源滤子可以作用于许多函数，也使同一个函数能在不同源滤子下讨论不同极限。

2.2 最基本的构造：principal 与 pure

2.2.1 主滤子 principal

给定 $S : \text{Set}(\alpha)$ ，主滤子

$$\text{principal}(S) : \text{Filter}(\alpha)$$

定义为

$$A \in \text{principal}(S) \iff S \subseteq A. \quad (5)$$

因此，命题沿 $\text{principal}(S)$ 最终成立，正是它在 S 的每一点成立。主滤子把普通集合限制嵌入 Filter 语言。

例 2.2：把定义域限制到集合

设 $P : \alpha \rightarrow \text{Prop}$ 。则

$$\forall^f x \text{ in } \text{principal}(S), P(x) \iff \forall x \in S, P(x).$$

例如沿 $\text{principal}([0, 1])$ ，命题 $x^2 \leq 1$ 最终成立；这里“最终”并不涉及时间，而是“在指定集合的全部点上”。

2.2.2 纯滤子 pure

对 $a : \alpha$ ，定义

$$\text{pure}(a) = \text{principal}(\{a\}) : \text{Filter}(\alpha).$$

于是

$$A \in \text{pure}(a) \iff a \in A. \quad (6)$$

沿 $\text{pure}(a)$ 最终成立就是在点 a 成立。这里完全不需要 α 上的拓扑。

例 2.3：函数作用于一个固定点

对任意 $f : \alpha \rightarrow \beta$ ，把 $\text{pure}(a)$ 经 f 推过去会得到 $\text{pure}(f(a))$ 。因此“固定输入 a 经函数变成固定输出 $f(a)$ ”也是一种退化但合法的趋近过程。稍后这会写成

$$\text{map } f(\text{pure}(a)) = \text{pure}(f(a)).$$

构造子的类型

```
open Filter Set
variable  $\alpha : \text{Type}^*$  ( $S : \text{Set } \alpha$ ) ( $a : \alpha$ )

#check (Filter.principal S : Filter  $\alpha$ )
#check (pure a : Filter  $\alpha$ )
#check mem_principal
#check mem_pure
```

2.3 滤子的序、顶底元与格运算**2.3.1 序**

Mathlib 在 $\text{Filter}(\alpha)$ 上定义

$$F \leq G \iff \forall A : \text{Set}(\alpha), A \in G \implies A \in F. \quad (7)$$

因此 $F \leq G$ 表示 F 至少满足 G 的全部最终条件；也可以说 F 比 G 更细、携带更多趋近信息。集合族的包含方向确实与符号相反：

$$F \leq G \iff G.\text{sets} \subseteq F.\text{sets}.$$

选择这个序的原因会在 Tendsto 中显现：极限恰好写成 $\text{map } fF \leq G$ ，复合则由序的传递性处理。

对主滤子，序又回到普通子集序：

$$\text{principal}(S) \leq \text{principal}(T) \iff S \subseteq T.$$

这可以直接由式 (5) 检查。

2.3.2 顶元与底元

滤子格的顶元与底元满足

$$\begin{aligned} A \in \top &\iff A = \alpha, & \top &= \text{principal}(\alpha), \\ A \in \perp &\iff \text{恒真}, & \perp &= \text{principal}(\emptyset). \end{aligned}$$

所以 $\top$ 只把全体集合视为最终区域，携带的信息最少； $\perp$ 把每个集合都视为最终区域，甚至包括空集，因而是退化滤子。

例 2.4: 顶底元对 eventually 的影响

对 $P : \alpha \rightarrow \text{Prop}$,

$$\forall^f x \text{ in } \top, P(x) \iff \forall x, P(x),$$

$$\forall^f x \text{ in } \perp, P(x) \iff \text{True}.$$

第一行是因为 $\{x : P(x)\}$ 必须等于全体；第二行是因为每个集合都属于 $\perp$ 。特别地，*False* 沿 $\perp$ 也最终成立。以后从 $\perp$ 出发的任意极限都真，这不是分析结论，而是源端没有可实际趋近的点。

2.3.3 交 $F \sqcap G$ 与并 $F \sqcup G$

下确界 $F \sqcap G$ 同时加入两种最终要求。其集合成员可表述为：

$$A \in F \sqcap G \iff \exists U \in F, \exists V \in G, U \cap V \subseteq A. \quad (8)$$

上确界则只保留两边共同认可的最终集合：

$$A \in F \sqcup G \iff A \in F \text{ 且 } A \in G. \quad (9)$$

对主滤子，这两条变成非常具体的公式：

$$\begin{aligned} \text{principal}(S) \sqcap \text{principal}(T) &= \text{principal}(S \cap T), \\ \text{principal}(S) \sqcup \text{principal}(T) &= \text{principal}(S \cup T). \end{aligned} \quad (10)$$

例 2.5: pure 的格运算

若 $a \neq b$ ，则

$$\text{pure}(a) \sqcap \text{pure}(b) = \text{principal}(\{a\} \cap \{b\}) = \perp.$$

“输入既固定为 a 又固定为 b ”是不相容的要求。另一方面

$$\text{pure}(a) \sqcup \text{pure}(b) = \text{principal}(\{a, b\}),$$

沿它最终成立表示命题在 a, b 两点都成立。这一例也解释了为什么滤子格必须容纳退化元素 $\perp$ ：自然的格运算会自动产生它。

2.3.4 完备格与条件完备格

一个**完备格**要求任意指标族都有上确界与下确界，包括空族；一个**条件完备格**只保证非空且具有相应界的集合存在上确界或下确界。 $\mathbb{R}$ 是典型的条件完备线性序：非空且有上界的实数集有上确界，但全体实数在 $\mathbb{R}$ 内没有上界，也没有上确界。

$\text{Filter}(\alpha)$ 是完备格。可以取任意族滤子的 $\bigwedge_i F_i$ 与 $\bigvee_i F_i$ ；空下确界为 $\top$ ，空上确界为 $\perp$ 。这使按任意条件限制滤子、构造聚点滤子、交换上下确界等操作都能无例外地陈述。

为什么允许退化滤子

若定义 Filter 时额外禁止 $\emptyset \in F$ ，自然的下确界可能不再存在， Filter 也不再是完备格。Mathlib 选择让所有格运算总有结果，并在需要“确实存在趋近对象”的定理处另加 `FNeBot`。这是把构造的总定义性与结论的非退化假设分开管理。

2.4 F.NeBot: 何时排除退化

F.NeBot (常以 `typeclass [F.NeBot]` 出现) 表示

$$F \neq \perp.$$

它等价于 $\emptyset \notin F$, 也等价于每个 F -最终集合都非空。对主滤子,

$$(\text{principal}(S)).\text{NeBot} \iff S \neq \emptyset. \quad (11)$$

例 2.6: 不可能的集合内趋近

在 $\mathbb{R}$ 中, 从集合 $\{1\}$ 内趋近 0 是不可能的。某个足够小的 0-邻域与 $\{1\}$ 不交, 所以相应滤子为 $\perp$ 。于是

$$\text{Tendsto } f(\mathcal{N}[\{1\}](0))G$$

对任意 f, G 都成立。若要从该极限推出极限唯一、抽取实际点或构造子列, 就必须假设源滤子 NeBot。

NeBot 与闭包

对拓扑空间中的 $S \subseteq X$,

$$(\mathcal{N}[S](a)).\text{NeBot} \iff a \in \text{cl}(S).$$

右边说每个 a 的邻域都与 S 相交, 正是“在 S 内趋近 a ”不退化的条件。

2.5 邻域滤子与集合内邻域

2.5.1 普通、集合内与去心邻域

在拓扑空间 X 中, a 的邻域滤子为

$$\mathcal{N}(a) : \text{Filter}(X).$$

给定 $S \subseteq X$, 集合内邻域的严格定义是

$$\mathcal{N}[S](a) = \mathcal{N}(a) \sqcap \text{principal}(S). \quad (12)$$

它同时施加“靠近 a ”与“位于 S 中”两个要求。Mathlib 的类型为 `nhdsWithin a S : Filter X`。

在带序拓扑中, 常用记号与展开如下:

纸面记号	Mathlib 对象	含义
$\mathcal{N}(a)$	<code>nhds a</code>	在 a 的某个邻域中
$\mathcal{N}[S](a)$	<code>nhdsWithin a S</code>	在 S 内趋近 a
$\mathcal{N}[\neq](a)$	<code>nhdsWithin a {a}^c</code>	靠近 a , 但忽略 a 本身
$\mathcal{N}[>](a)$	<code>nhdsWithin a (Set.Ioi a)</code>	从严格大于 a 的一侧趋近
$\mathcal{N}[<](a)$	<code>nhdsWithin a (Set.Iio a)</code>	从严格小于 a 的一侧趋近

例 2.7: 平方根在定义域内连续

$x \mapsto \sqrt{x}$ 自然定义在 $[0, +\infty)$ 。在端点 0 讨论连续性时，源端应使用

$$\mathcal{N}_{[0, +\infty)}(0),$$

而不是先把函数任意延拓到全部实数。`ContinuousWithinAt` 与 `nhdsWithin` 正是用来陈述这种内在于定义域的局部性质。

例 2.8: 去心邻域区分极限与点值

若 $f, g: \mathbb{R} \rightarrow \mathbb{R}$ 只在 a 处取值不同，那么

$$f =^f [\mathcal{N}[\neq](a)]g.$$

因此二者在 $x \rightarrow a$, $x \neq a$ 时具有完全相同的极限行为；但它们在 a 的连续性可以不同。`Filter` 参数精确记录了哪些输入允许被忽略。

2.6 无穷方向: `atTop` 与 `atBot`

若 α 带有适当的序结构，`atTop: Filter( $\alpha$ )` 由上尾集生成，`atBot: Filter( $\alpha$ )` 由下尾集生成。在 $\mathbb{N}$ 或 $\mathbb{R}$ 上可写为

$$A \in \text{atTop} \iff \exists a, \{x : a \leq x\} \subseteq A, \quad (13)$$

$$A \in \text{atBot} \iff \exists a, \{x : x \leq a\} \subseteq A. \quad (14)$$

例 2.9: 充分大的自然数

命题 $n^2 \geq 100$ 沿 `atTop` 最终成立，因为从 $N = 10$ 开始它恒真。奇数性既不最终成立，也不最终不成立：任何尾部里同时有奇数和偶数。后一事实将由 `Frequently` 表达。

例 2.10: 趋向负无穷

对实变量 x ，命题 $x^3 < -10^6$ 沿 `atBot` 最终成立。函数极限 $f(x) \rightarrow L$ ($x \rightarrow -\infty$) 统一写作

$$\text{Tendsto } f \text{ atBot } \mathcal{N}(L).$$

无需另定义“负无穷处的邻域”。

2.6.1 再比较扩充数与 `atTop`

“趋于无穷”有两种建模方式，不能只看记号而不看类型。第一种是在原类型上指定一个方向：若 α 带有适当的序结构，`atTop: Filter( $\alpha$ )` 仍然定义在 α 上，表达“超过任意给定界”；它不向类型中添加新的点。第二种是扩大值域，把无穷作为一个真正的点，再使用该点的邻域滤子。`Mathlib` 中最常见的类型如下：

Lean 类型	数学对象	典型用途
ENat ($\mathbb{N}_\infty$)	$\mathbb{N} \cup \{\infty\}$	允许无穷的自然数值、基数与次数
ENNReal ($\mathbb{R}_{\geq 0\infty}$)	$[0, +\infty]$	测度、非负积分及可能为无穷的大小
EReal ($\overline{\mathbb{R}}$)	$[-\infty, +\infty]$	同时记录正、负无穷的极限与凸分析

抽象地看, $\mathbb{N}_\infty = \text{WithTop}(\mathbb{N})$, $\mathbb{R}_{\geq 0\infty} = \text{WithTop}(\mathbb{R}_{\geq 0})$, 而 $\overline{\mathbb{R}} = \text{WithBot}(\text{WithTop}(\mathbb{R}))$ 。这些构造添加的是值域中的点; `atTop` 则只是原类型上的一个滤子。

例如自然数嵌入扩充自然数后, 极限可以写成

$$\text{Tendsto}(n \mapsto (n : \mathbb{N}_\infty)) \text{ atTop } \mathcal{N}(\top : \mathbb{N}_\infty).$$

而对实值函数 $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) \rightarrow +\infty$ 只能写成

$$\text{Tendsto } f \text{ F atTop},$$

因为 $+\infty \notin \mathbb{R}$ 。若函数值本来在 $\mathbb{R}_{\geq 0\infty}$ 中, 则可以把同一现象写成

$$\text{Tendsto } g \text{ F } \mathcal{N}(\top : \mathbb{R}_{\geq 0\infty}),$$

此时无穷点参与值域中的运算。Mathlib 提供相应的互译定理; 例如 `ENat.tendsto_nhds_top_iff_natCast_lt` 和 `ENNReal.tendsto_nhds_top_iff_nat` 把邻域陈述展开为“最终超过每个有限界”。数列索引通常采用 `atTop`, 而测度值采用 $\mathbb{R}_{\geq 0\infty}$, 正是因为两者承担的角色不同。

扩充值域的类型查询

```
#check ENat.tendsto_nhds_top_iff_natCast_lt
#check ENNReal.tendsto_nhds_top_iff_nat
#check ENNReal.tendsto_nat_nhds_top
#check EReal.tendsto_coe_nhds_top_iff
```

2.7 Eventually: 滤子的命题语言

2.7.1 定义与基本推理

给定谓词 $P : \alpha \rightarrow \text{Prop}$ 与 $F : \text{Filter}(\alpha)$, Mathlib 定义

$$\text{Eventually}(P, F) : \text{Prop}$$

为

$$\forall^f x \text{ in } F, P(x) \iff \{x : P(x)\} \in F. \quad (15)$$

Lean 记号是 $\forall^f x \text{ in } F, P x$ 。滤子的三个公理立即变成以下推理规则: 全称真命题最终成立; 最终命题可由蕴含放宽; 两个最终命题可合取。

Eventually 的类型与常用规则

```
#check Filter.Eventually -- (α → Prop) → Filter α → Prop
#check Filter.Eventually.of_forall
#check Filter.Eventually.mono
#check Filter.Eventually.and
```

-- filter_upwards is tactic syntax for combining eventual hypotheses.

不同滤子下，式 (15) 恢复熟悉的说法：

F	$\forall^f x \text{ in } F, P(x)$ 的含义
$\text{pure}(a)$	$P(a)$
$\text{principal}(S)$	对每个 $x \in S$, $P(x)$
$\mathcal{N}(a)$	P 在 a 的某个邻域内成立
$\mathcal{N}[\neq](a)$	P 在 a 的某个去心邻域内成立
atTop	存在阈值 N , 此后 P 恒成立
$\top$	对所有 x , $P(x)$
$\perp$	无条件为真, 包括 $P = \text{False}$

例 2.11: 最终估计的合取

设 $u_n \rightarrow L$ 。给定 $\varepsilon > 0$, 证明中可能分别得到

$$\forall^f n \text{ in } \text{atTop}, |u_n - L| < 1, \quad \forall^f n \text{ in } \text{atTop}, |u_n - L| < \varepsilon/2.$$

滤子公理允许直接进入一个两者同时成立的尾部。Lean 的 filter_upwards 正是把若干 eventually 假设合并, 并在共同最终区域内做逐点推理。

例 2.12: 邻域中的最终性质

在 $\mathbb{R}$ 中, $|x| < 1$ 沿 $\mathcal{N}(0)$ 最终成立; $0 < |x| < 1$ 沿 $\mathcal{N}[\neq](0)$ 最终成立。这两条只是 “ $(-1, 1)$ 是 0 的邻域” 和 “在去心邻域中 $x \neq 0$ ” 的直接翻译。

2.8 Frequently: 无法被最终排除

Frequently 的类型同样是

$$(\alpha \rightarrow \text{Prop}) \rightarrow \text{Filter}(\alpha) \rightarrow \text{Prop},$$

定义为

$$\exists^f x \text{ in } F, P(x) \iff \neg(\forall^f x \text{ in } F, \neg P(x)). \quad (16)$$

Lean 记号是 $\exists^f x \text{ in } F, P x$ 。一般滤子下, frequently 最准确的读法是 “ P 不能在最终区域中被排除”; 只有有在 atTop 等具体滤子下, 它才对应 “任意靠后的区域里仍会发生”。

例 2.13: 偶数在 atTop 中 frequently 成立

对每个阈值 N , 都能找到偶数 $n \geq N$ 。因此

$$\exists^f n \text{ in } \text{atTop}, 2 \mid n.$$

同时偶数性并非 eventually, 因为任意尾部也含奇数。相反, 条件 $n < 100$ 不 frequently, 因为从 100 开始它被永久排除。

闭包的 frequently 刻画

对拓扑空间 X 、集合 $S \subseteq X$ 与点 $x \in X$,

$$x \in \text{cl}(S) \iff \exists^f y \text{ in } \mathcal{N}(x), y \in S. \quad (17)$$

右边说不存在一个 x 的邻域完全避开 S 。例如 $0 \in \text{cl}((0,1))$ ，因为每个 0 的邻域都含有 $(0,1)$ 中的点。注意左边是 $x \in \text{cl}(S)$ ，不是 $x \in S$ 。

顶底元再次给出有用的边界检查：沿 $\top$ ，frequently P 等价于存在某个 x 使 $P(x)$ ；沿 $\perp$ ，没有命题 frequently 成立，因为其否定总是 eventually。

2.9 map 与 comap：沿函数搬运滤子

2.9.1 map：推送源端行为

给定 $f : \alpha \rightarrow \beta$ 与 $F : \text{Filter}(\alpha)$ ，定义

$$\text{map}(f, F) : \text{Filter}(\beta)$$

使

$$B \in \text{map}(f, F) \iff f^{-1}(B) \in F. \quad (18)$$

$\text{map}(f, F)$ 记录“当输入沿 F 变化时，输出最终落入哪些集合”。它是函数值本身诱导出的目标端行为。

例 2.14：map 一条数列

若 $u : \mathbb{N} \rightarrow X$ ，则 $\text{map}(u, \text{atTop})$ 是 X 上的滤子，集合 $S \subseteq X$ 属于它当且仅当 u_n 最终落在 S 中。数列收敛到 L ，就是这个像滤子比 L 的邻域滤子更细。

例 2.15：map 保持复合

对 $f : \alpha \rightarrow \beta$ 、 $g : \beta \rightarrow \gamma$ ，

$$\text{map}(g, \text{map}(f, F)) = \text{map}(g \circ f, F).$$

因为一个集合经两次原像等于经复合函数的一次原像。连续映射复合与极限复合最终都来自这条结合律。

2.9.2 comap：拉回目标条件

给定 $f : \alpha \rightarrow \beta$ 与 $G : \text{Filter}(\beta)$ ，定义

$$\text{comap}(f, G) : \text{Filter}(\alpha).$$

它由所有 $f^{-1}(B)$ ($B \in G$) 生成：严格地，

$$A \in \text{comap}(f, G) \iff \exists B \in G, f^{-1}(B) \subseteq A. \quad (19)$$

因此 $\text{comap}(f, G)$ 是“为了让函数值沿 G ，源端至少需要满足的条件”。

例 2.16: 由一个观测量定义趋近

设 $p: X \rightarrow \mathbb{R}$ 是某个大小函数。滤子 $\text{comap}(p, \mathcal{N}(0))$ 表示 “ $p(x) \rightarrow 0$ ” 所诱导的源端趋近：一个集合 $A \subseteq X$ 最终成立，只要存在 0 的邻域 V ，使 $p^{-1}(V) \subseteq A$ 。例如取 $p(x) = \|x\|$ ，它刻画由范数趋于零所控制的源端行为。

2.9.3 map-comap 伴随与复合

两者满足关键等价式

$$\text{map}(f, F) \leq G \iff F \leq \text{comap}(f, G). \quad (20)$$

这是一个 Galois 连接：把输入行为推到值域后足以满足 G ，等价于输入行为本身足以满足 G 的拉回条件。另有

$$\text{comap}(f, \text{comap}(g, H)) = \text{comap}(g \circ f, H).$$

所以 map 与 comap 都把函数复合转化为滤子上的组合。

直觉与使用边界

map 回答 “沿 F 观察 f ，会在值域看到什么”； comap 回答 “若希望 f 的值满足 G ，定义域中哪些条件已经足够”。前者直接进入极限定义，后者常用于诱导拓扑、子类型、变量替换和构造源滤子。二者的正式定义都以原像为核心，因为原像保持任意并与有限交。

map 与 comap 的类型

variable $\alpha \beta : \text{Type}^* (f : \alpha \rightarrow \beta)$

```
#check Filter.map      -- (α → β) → Filter α → Filter β
#check Filter.comap    -- (α → β) → Filter β → Filter α
#check Filter.map_mono
#check Filter.map_map
#check Filter.map_le_iff_le_comap
```

2.10 EventuallyEq: 最终相等与局部数学**2.10.1 严格定义**

对 $f, g : \alpha \rightarrow \beta$ 与 $F : \text{Filter}(\alpha)$ ，定义

$$f =^f [F] g \iff \forall^f x \text{ in } F, f(x) = g(x). \quad (21)$$

Lean 记作 $f =^f [F] g$ ，其类型是一个命题。EventuallyEq 是自反、对称、传递的等价关系，并且与函数复合及逐点运算相容。

例 2.17: 忽略数列的有限开头

定义 $f, g : \mathbb{N} \rightarrow \mathbb{R}$:

$$f(n) = \begin{cases} 1000, & n = 0, \\ 1/n, & n > 0, \end{cases} \quad g(n) = \begin{cases} 0, & n = 0, \\ 1/n, & n > 0. \end{cases}$$

从 $n \geq 1$ 开始二者完全相等，所以

$$f =^f [\text{atTop}]g.$$

任何只依赖 $n \rightarrow \infty$ 行为的命题都不应区分它们，特别是二者有相同极限。

2.10.2 在邻域滤子下：局部相等

$$f =^f [\mathcal{N}(a)]g$$

表示存在 a 的某个邻域，在整个邻域中 $f = g$ ；而

$$f =^f [\mathcal{N}[\neq](a)]g$$

表示在 a 附近忽略 a 本身后相等。后一关系足以保证二者在 $x \rightarrow a$, $x \neq a$ 时具有相同极限，却不强迫 $f(a) = g(a)$ 。

Mathlib 的 `tendsto_congr'`、`Filter.Tendsto.congr'` 等定理把最终相等与极限直接连接；当函数最终等于常值时，还可使用 `Filter.EventuallyEq.tendsto`。因此可以先在最终区域把复杂函数替换成较简单函数，再证明后者的极限。

例 2.18：可去奇点

令

$$f(x) = \frac{x^2 - 1}{x - 1}, \quad g(x) = x + 1.$$

f 在 $x = 1$ 的表达式含除零，但在 $\mathcal{N}[\neq](1)$ 上有 $f =^f [\mathcal{N}[\neq](1)]g$ 。因此 $f(x) \rightarrow 2$ 可由 g 的连续性得到。形式化证明不必反复把“ $x \neq 1$ ”带进每个代数步骤；只需在最终区域化简一次。

2.10.3 函数芽 germ

拓扑学与微分几何把在 a 的某个邻域中相等的函数视作具有相同局部行为。定义

$$f \sim_a g \iff f =^f [\mathcal{N}(a)]g,$$

并对这一等价关系取商，所得等价类称为 a 处的**函数芽** (germ)。函数芽忘掉远离 a 的全部信息，只保留任意充分小邻域内稳定的行为。导数、局部环、解析函数芽都建立在这种局部化思想上。

因此 `EventuallyEq` 并非用于规避边界情况的工程工具；它直接实现了“局部对象按邻域相等”的数学原则。Mathlib 还提供以滤子参数化的 `Filter.Germ` 相关结构，使这一思想不局限于点邻域。

2.11 EventuallyLE 与渐近分析

2.11.1 最终不等式

若值域带序，定义

$$f \leq^f [F]g \iff \forall^f x \text{ in } F, f(x) \leq g(x). \quad (22)$$

Lean 记作 `f ≤f[F] g`。它是 `Eventually` 作用于二元不等式后的记号，不要求不等式在全部定义域成立。

例 2.19: 最终支配

对自然数 n , $3n + 7 \leq n^2$ 并非处处成立, 但从某个阈值开始恒成立, 故

$$(n \mapsto 3n + 7) \leq^f [\text{atTop}](n \mapsto n^2).$$

在邻域滤子下, $f \leq^f [\mathcal{N}(a)]g$ 则表示存在 a 的某个邻域, 使其中每一点都有 $f(x) \leq g(x)$ 。极限中的夹逼、最终正性与误差估计都使用同一种关系。

2.11.2 Big-O 把滤子作为趋近参数

对取值于赋范空间的函数, Mathlib 的渐近记号

$$f = O[F]g$$

表示存在常数 $C > 0$, 使

$$\forall x \text{ in } F, \quad \|f(x)\| \leq C\|g(x)\|. \quad (23)$$

Lean 记作 $f = O[F]g$, 正式命题名是 `Asymptotics.IsBigO`。关键是估计只需在 F 关心的最终区域成立。

例 2.20: 无穷远处的 Big-O

令 $f(n) = 3n^2 + n + 5$, $g(n) = n^2$ 。对充分大的 n 可取某个常数 C 使 $|f(n)| \leq C|g(n)|$, 所以

$$f = O[\text{atTop}]g.$$

展开后正是熟悉的“存在 C, N , 对所有 $n \geq N$ 成立”。有限多个例外不会改变结论。

例 2.21: 局部 Big-O

不等式 $|\sin x| \leq |x|$ 给出

$$\sin = O[\mathcal{N}(0)](x \mapsto x).$$

若把滤子改为 $\mathcal{N}[\neq](0)$, 就表示 $x \rightarrow 0$ 且 $x \neq 0$ 时的局部估计。渐近等价、little-o 与 Big-O 都以 Filter 为参数, 因此同一套代数定理可同时服务于 $x \rightarrow a$ 、 $x \rightarrow \infty$ 和数列指标趋于无穷。

这正是 Filter 抽象开始产生实际收益的地方: 一旦“最终区域”成为参数, 渐近估计的传递、乘法、复合和换元不需要为每一种极限方向重写。

2.12 Tendsto: 统一极限的一个关系**2.12.1 严格定义**

给定

$$f : \alpha \rightarrow \beta, \quad F : \text{Filter}(\alpha), \quad G : \text{Filter}(\beta),$$

Mathlib 定义

$$\text{Tendsto } f F G \iff \text{map}(f, F) \leq G. \quad (24)$$

展开滤子序与 map, 得到完全等价的原像表述:

$$\text{Tendsto } f F G \iff \forall B \in G, f^{-1}(B) \in F. \quad (25)$$

也就是说, 目标端每个最终要求, 经 f 拉回以后都在源端最终成立。

Tendsto 的类型

$$\text{Tendsto} : (\alpha \rightarrow \beta) \rightarrow \text{Filter}(\alpha) \rightarrow \text{Filter}(\beta) \rightarrow \text{Prop}.$$

函数、源滤子与目标滤子的类型已经决定命题的解释。 F 说明输入如何变化， G 说明输出应满足何种趋近要求。

定义与基本定理

```
#check Filter.Tendsto
#check tendsto_def
#check tendsto_map'_iff
#check Filter.Tendsto.comp
#check tendsto_id
#check tendsto_const_nhds
```

2.12.2 三个最小例子

1. 恒等函数满足 $\text{Tendsto id } F F$ ，因为 $\text{map}(\text{id}, F) = F$ 。
2. 任意函数满足 $\text{Tendsto } f \text{ pure}(a) \text{ pure}(f(a))$ ，因为固定点被送到固定点。
3. 若 g 沿 G 趋于 H ， f 沿 F 趋于 G ，则 $g \circ f$ 沿 F 趋于 H ：

$$\text{Tendsto } g G H \wedge \text{Tendsto } f F G \implies \text{Tendsto } (g \circ f) F H. \quad (26)$$

证明只是 map 的复合律与 $\leq$ 的传递性。

例 2.22: 连续性已经包含在 Tendsto 中

取 $F = \mathcal{N}(a)$ 、 $G = \mathcal{N}(f(a))$ ，式 (25) 变为：对 $f(a)$ 的每个邻域 V ， $f^{-1}(V)$ 是 a 的邻域。这正是 $\text{ContinuousAt } f a$ 。因此连续映射的复合只是式 (26) 的一个实例。

2.12.3 为什么 Mathlib 选择 Filter

现在可以给出具体回答。

1. 消除定义复制。数列、函数、单侧、去心、无穷远和集合内极限只更换 F, G ，核心关系不变。
2. 复合在结构层完成。原像复合、 map 复合和序传递直接给出极限复合；无需再次追踪 ε, δ, N 。
3. 最终推理成为独立接口。EventuallyEq、EventuallyLE、Big-O 与几乎处处都复用滤子的逻辑。
4. 退化情况被显式管理。完备格保证构造总有定义，NeBot 在需要时排除空洞结论。
5. 类型显示结构依赖。一个定理若只谈 Tendsto ，往往只需拓扑；误差界则会额外要求度量或范数。形式化接口与数学假设同步。

阅读 Tendsto 的固定顺序

看到 $\text{Tendsto } f F G$ 时，依次问：(1) f 的定义域和值域是什么；(2) F 允许忽略哪些输入；(3) G 把哪些输出条件视为最终要求；(4) 目标条件的原像为什么属于 F 。按这个顺序，复杂的 Mathlib 极限定理通常可以还原成一条熟悉的分析陈述。

3 第三章 极限与连续性的形式化

本章不再引入新的极限定义，而是为 `Tendsto` 选择不同的源滤子和目标滤子。这样做的价值不只在缩短记号：不同极限共享复合、同余、唯一性与代数运算定理，证明可以在统一接口上组合。

3.1 极限词典：只替换滤子参数

设 X, Y 为拓扑空间， $f : X \rightarrow Y$ ， $a \in X$ ， $L \in Y$ ；设 $u : \mathbb{N} \rightarrow Y$ 。常见分析命题对应如下：

传统读法	Filter 表述	Mathlib 源端
$x \rightarrow a$ 时 $f(x) \rightarrow L$ ，忽略点值	$\text{Tendsto } f \mathcal{N}[\neq](a) \mathcal{N}(L)$	<code>nhdsWithin a {a}ᶜ</code>
$x \rightarrow a$ 时 $f(x) \rightarrow f(a)$	$\text{Tendsto } f \mathcal{N}(a) \mathcal{N}(f(a))$	<code>nhds a</code>
$x \rightarrow a$ 且 $x \in S$ 时 $f(x) \rightarrow L$	$\text{Tendsto } f \mathcal{N}[S](a) \mathcal{N}(L)$	<code>nhdsWithin a S</code>
$x \rightarrow a^+$ 时 $f(x) \rightarrow L$	$\text{Tendsto } f \mathcal{N}[>](a) \mathcal{N}(L)$	<code>nhdsWithin a (Set.Ioi a)</code>
$x \rightarrow a^-$ 时 $f(x) \rightarrow L$	$\text{Tendsto } f \mathcal{N}[<](a) \mathcal{N}(L)$	<code>nhdsWithin a (Set.Iio a)</code>
数列 $u_n \rightarrow L$	$\text{Tendsto } u \text{ atTop } \mathcal{N}(L)$	<code>atTop : Filter ℕ</code>
$x \rightarrow +\infty$ 时 $f(x) \rightarrow L$	$\text{Tendsto } f \text{ atTop } \mathcal{N}(L)$	<code>atTop : Filter ℝ</code>
$x \rightarrow -\infty$ 时 $f(x) \rightarrow L$	$\text{Tendsto } f \text{ atBot } \mathcal{N}(L)$	<code>atBot : Filter ℝ</code>
沿 F 有 $f(x) \rightarrow +\infty$	$\text{Tendsto } f F \text{ atTop}$	目标滤子 <code>atTop</code>
沿 F 有 $f(x) \rightarrow -\infty$	$\text{Tendsto } f F \text{ atBot}$	目标滤子 <code>atBot</code>

同一个 `atTop` 可以出现在源端或目标端，但含义由类型与位置决定：

$$\text{Tendsto } u \underbrace{\text{atTop}_{\mathbb{N}}}_{\text{指标充分大}} \mathcal{N}(L), \quad \text{Tendsto } f F \underbrace{\text{atTop}_{\mathbb{R}}}_{\text{函数值超过任意界}}.$$

这也是为什么阅读 Lean 陈述时必须先看类型。

3.2 ε - δ 与 `Tendsto` 的等价

3.2.1 目标为度量邻域时先展开 `epsilon`

设 Y 为度量空间， $f : \alpha \rightarrow Y$ ， $F : \text{Filter}(\alpha)$ 。目标邻域有开球基，因此

$$\text{Tendsto } f F \mathcal{N}(b) \iff \forall \varepsilon > 0, \forall^f x \text{ in } F, d_Y(f(x), b) < \varepsilon. \quad (27)$$

这一步只展开目标滤子，源端 F 仍保持抽象。它适用于数列、点邻域、无穷远以及测度论中的几乎处处滤子。

例 3.1：数列的 `epsilon-N` 定义

在式 (27) 中取 $F = \text{atTop}$ 、 $Y = \mathbb{R}$ ，得到

$$\forall \varepsilon > 0, \forall^f n \text{ in } \text{atTop}, |u_n - L| < \varepsilon.$$

再展开 atTop 的尾集基，就是

$$\forall \varepsilon > 0, \exists N, \forall n \geq N, |u_n - L| < \varepsilon.$$

Filter 定义没有取代 ε - N ；它把定义分成“目标球基”和“源尾集基”两次展开。

3.2.2 源也为度量邻域时再展开 delta

若 X 也是度量空间，则

$$\varepsilon\text{-}\delta \iff \text{Tendsto}$$

对 $f : X \rightarrow Y$ 与 $a \in X, b \in Y$,

$$\begin{aligned} & \text{Tendsto } f \mathcal{N}(a) \mathcal{N}(b) \\ & \iff \forall \varepsilon > 0, \exists \delta > 0, \forall x, d_X(x, a) < \delta \implies d_Y(f(x), b) < \varepsilon. \end{aligned} \quad (28)$$

若将源端换为去心邻域，则

$$\begin{aligned} & \text{Tendsto } f \mathcal{N}[\neq](a) \mathcal{N}(b) \\ & \iff \forall \varepsilon > 0, \exists \delta > 0, \forall x, 0 < d_X(x, a) < \delta \implies d_Y(f(x), b) < \varepsilon. \end{aligned} \quad (29)$$

证明式 (28) 时，正向先给定目标开球 $B_Y(b, \varepsilon)$ 。由 Tendsto ，其原像是 a 的邻域；由度量邻域的开球基，存在 $\delta > 0$ 使

$$B_X(a, \delta) \subseteq f^{-1}(B_Y(b, \varepsilon)).$$

这就是 ε - δ 蕴含式。反向则对目标端任一邻域 V ，先取 $\varepsilon > 0$ 使 $B_Y(b, \varepsilon) \subseteq V$ ，再用假设取得 δ ；于是 $f^{-1}(V)$ 含有 $B_X(a, \delta)$ ，故为 a 的邻域。

去心版本的证明完全相同，只把源球替换为 $B_X(a, \delta) \setminus \{a\}$ 。所以 Filter 语言并没有隐藏量词，而是将量词组织进可复用的邻域基定理。

Mathlib 中的展开定理

```
variable α β : Type* [MetricSpace α] [MetricSpace β]
variable f : α → β a : α b : β l : Filter α
```

```
#check Metric.tendsto_nhds
#check Metric.tendsto_nhds_nhds
#check Metric.continuousAt_iff
```

何时展开，何时保持抽象

构造具体 δ 、证明数值误差界时，展开度量邻域通常最直接；证明复合、同余、连续函数保极限时，保持 Tendsto 形式更稳定。良好的形式化证明会在结构层尽量组合，只在真正需要估计的局部步骤下降到 ε - δ 。

3.3 点极限、无穷远与数列极限

3.3.1 普通点与去心点

传统记号 $\lim_{x \rightarrow a} f(x) = L$ 通常忽略 $f(a)$, 故源端应为 $\mathcal{N}[\neq](a)$ 。如果教材约定 f 只定义在 $S \setminus \{a\}$, 则更精确的源端是

$$\mathcal{N}[S \setminus \{a\}](a).$$

这避免先任意延拓函数, 再证明结论与延拓无关。

例 3.2: 最终相等转移极限

沿用可去奇点函数

$$f(x) = \frac{x^2 - 1}{x - 1}, \quad g(x) = x + 1.$$

有 $f =^f [\mathcal{N}[\neq](1)]g$, 而 g 在 1 连续, 所以

$$\text{Tendsto } g \mathcal{N}[\neq](1) \mathcal{N}(2).$$

由 eventual equality 的同余定理立即得到 f 的同一极限。这里先处理局部代数事实, 再调用抽象极限定理; 两类推理彼此分离。

3.3.2 数列只是定义域为自然数的函数

数列 $u : \mathbb{N} \rightarrow X$ 的收敛写作

$$\text{Tendsto } u \text{ atTop } \mathcal{N}(L).$$

它不需要单独的“序列极限”结构。若 $\varphi : \mathbb{N} \rightarrow \mathbb{N}$ 严格单调且趋于无穷, 则子列 $u \circ \varphi$ 的收敛由复合得到:

$$\text{Tendsto } \varphi \text{ atTop atTop}, \quad \text{Tendsto } u \text{ atTop } \mathcal{N}(L) \implies \text{Tendsto}(u \circ \varphi) \text{ atTop } \mathcal{N}(L).$$

例 3.3: 修改有限项不改变数列极限

若 $u =^f [\text{atTop}]v$, 则

$$\text{Tendsto } u \text{ atTop } \mathcal{N}(L) \iff \text{Tendsto } v \text{ atTop } \mathcal{N}(L).$$

“删掉有限项不改变收敛性”不再需要关于有限集的专门引理, 它是 `tendsto_congr` 在 `atTop` 上的实例。

3.3.3 源趋于无穷与值趋于无穷

对 $f : \mathbb{R} \rightarrow \mathbb{R}$,

$$\text{Tendsto } f \text{ atTop } \mathcal{N}(L)$$

表示 $x \rightarrow +\infty$ 时 $f(x) \rightarrow L$ 。若目标也换成 `atTop`, 则

$$\text{Tendsto } f F \text{ atTop} \iff \forall M \in \mathbb{R}, \forall^f x \text{ in } F, M \leq f(x), \quad (30)$$

即函数值最终超过任意给定界。

例 3.4: $x^2 \rightarrow +\infty$

在源端取 $\text{atTop}_{\mathbb{R}}$ 。给定 $M \in \mathbb{R}$ ，选择 $A > \max(0, \sqrt{\max(M, 0)})$ ，则 $x \geq A$ 蕴含 $x^2 \geq M$ 。所以

$$\text{Tendsto}(x \mapsto x^2) \text{ atTop atTop}.$$

如果源端改成 $\mathcal{N}[\neq](0)$ 、函数改成 $x \mapsto 1/x^2$ ，相同目标滤子表达 $x \rightarrow 0$ 时函数趋于 $+\infty$ 。目标概念不变，只有源端变化。

例 3.5: 扩充值域中的同一结论

设 $g(x) = \text{ENNReal.ofReal}(x^2)$ 。可以把结论写成

$$\text{Tendsto } g \text{ atTop}_{\mathbb{R}} \mathcal{N}(+\infty).$$

Mathlib 关于 `ENNReal.tendsto_nhds_top_iff_nat` 等定理把它展开为“最终超过每个有限自然数界”。这与式 (30) 同构，但目标类型中现在允许函数实际取 $+\infty$ 。

3.4 左极限、右极限与两侧合并

在 $\mathbb{R}$ 中定义

$$F_- := \mathcal{N}[<](a), \quad F_+ := \mathcal{N}[>](a).$$

右极限与左极限分别是 $\text{Tendsto } f F_+ \mathcal{N}(L)$ 与 $\text{Tendsto } f F_- \mathcal{N}(L)$ 。由于实线性序中去心邻域由左右两侧覆盖，

$$\mathcal{N}[\neq](a) = F_- \sqcup F_+. \quad (31)$$

而 map 保持上确界，所以

$$\begin{aligned} \text{Tendsto } f \mathcal{N}[\neq](a) \mathcal{N}(L) &\iff \text{Tendsto } f F_- \mathcal{N}(L) \\ &\quad \wedge \text{Tendsto } f F_+ \mathcal{N}(L). \end{aligned} \quad (32)$$

这就是“两侧极限存在且相等，当且仅当普通极限存在”的滤子格解释。

例 3.6: 绝对值商的两侧极限

对 $s(x) = |x|/x$ ($x \neq 0$),

$$\text{Tendsto } s \mathcal{N}[>](0) \mathcal{N}(1), \quad \text{Tendsto } s \mathcal{N}[<](0) \mathcal{N}(-1).$$

若普通去心极限存在，在 Hausdorff 目标空间 $\mathbb{R}$ 中必须同时等于 1 和 -1，矛盾。所以 s 在 0 没有两侧极限。Filter 形式把“从哪一侧来”放在假设中，而不改函数表达式。

3.5 连续性: ContinuousAt 及其组合**3.5.1 点态连续的定义**

对拓扑空间 X, Y 与 $f : X \rightarrow Y$ ，Mathlib 定义

$$\text{ContinuousAt}(f, a) \iff \text{Tendsto } f \mathcal{N}(a) \mathcal{N}(f(a)). \quad (33)$$

对应的层级还有：

$$\begin{aligned} \text{ContinuousWithinAt}(f, S, a) &\iff \text{Tendsto } f \mathcal{N}[S](a) \mathcal{N}(f(a)), \\ \text{ContinuousOn}(f, S) &\iff \forall a \in S, \text{ContinuousWithinAt}(f, S, a), \\ \text{Continuous}(f) &\iff \forall a, \text{ContinuousAt}(f, a). \end{aligned}$$

这些定义把局部、集合内与全局连续性连接成一个清晰层级。

3.5.2 连续映射复合

若

$$h_f : \text{ContinuousAt}(f, a), \quad h_g : \text{ContinuousAt}(g, f(a)),$$

则

$$h_g \cdot \text{comp}(h_f) : \text{ContinuousAt}(g \circ f, a).$$

其证明就是

$$\mathcal{N}(a) \xrightarrow{f} \mathcal{N}(f(a)) \xrightarrow{g} \mathcal{N}(g(f(a)))$$

对应的两次 `Tendsto` 复合。与手工 ε - δ 证明相比，中间精度的选择已经封装在两个连续性假设中。

连续性的结构化 API

```
variable X Y Z : Type*
variable [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z]
variable f : X → Y g : Y → Z x : X

#check ContinuousAt
#check ContinuousWithinAt
#check ContinuousOn
#check Continuous
#check ContinuousAt.comp

example (hg : ContinuousAt g (f x)) (hf : ContinuousAt f x) :
  ContinuousAt (g ∘ f) x := hg.comp hf
```

例 3.7：由组合子证明多项式连续

恒等函数与常函数连续；连续实函数的和、积仍连续。因此

$$x \mapsto x^3 + 2x + 1$$

的连续性由 `continuousAt_id.pow`、`add`、`mul` 和 `continuousAt_const` 组合得到。证明不展开极限定义，因为这些代数运算的连续性已经以可组合定理存在。

3.6 极限证明的三种层次

形式化时可按需要停留在不同层次。

1. **结构层**：直接组合已有的 `Tendsto`、`ContinuousAt`、`EventuallyEq` 定理。复合函数、代数运算和有限修改通常在此层完成。

2. **eventually 层**: 把目标极限展开成“对任意目标条件, 某性质最终成立”, 用 `filter_upwards` 合并最终估计。夹逼与最终正性常在此层完成。
3. **基层**: 把邻域或 `atTop` 展开为开球、区间、尾集, 显式构造 δ 或 N 。新数值估计通常必须下降到这一层。

例 3.8: 夹逼的 `eventually` 形态

若沿 F 最终有 $g(x) \leq f(x) \leq h(x)$, 并且 g, h 都沿 F 趋于 L , 则在有序拓扑的适当假设下 f 也趋于 L 。传统证明先为 g, h 各找一个阈值, 再取较大者; `Filter` 证明用最终合取自动进入共同区域, 再调用序闭性。

3.7 唯一性为何需要结构条件

若 $F = \perp$, 任意 L 都是任意函数沿 F 的极限; 所以极限唯一至少需要 $F.\text{NeBot}$ 。即使源端非退化, 若目标拓扑不能区分不同点, 极限仍可能不唯一。因此标准唯一性定理要求:

$$F.\text{NeBot}, \quad Y \text{ 是 Hausdorff (Mathlib: T2Space Y)}.$$

在这些假设下,

$$\text{Tendsto } f F \mathcal{N}(a), \quad \text{Tendsto } f F \mathcal{N}(b) \implies a = b.$$

例 3.9: 为什么普通点连续不需额外写 `NeBot`

$\mathcal{N}(a)$ 总是非退化, 因为每个 a 的邻域都含 a 。所以连续性源端自动具有实际内容。去心滤子 $\mathcal{N}[\neq](a)$ 却可能在孤立点处等于 $\perp$; 讨论去心极限唯一时不能省略这一差别。`Mathlib` 的假设正是在这里防止空洞结论。

前三章的结论

ε - δ 是度量邻域基的坐标表达; 拓扑把坐标表达提升为邻域原像; `Filter` 再把邻域与其他“最终性”结构放入同一类型。抽象没有改变极限的数学内容, 而是把反复出现的量词模式变成可组合对象。后面的微分与积分会继续使用同一策略: 先找出稳定的结构接口, 再在接口上组织存在性和组合定理。

4 第四章 微分：从极限到 Fréchet 导数

Mathlib 不把一元差商作为微分理论的基础。基础对象是赋范空间之间的 Fréchet 导数：一个在给定点处提供最佳一阶近似的连续线性映射。一元导数是这一结构在一维空间上的特例。

4.1 导数是局部线性逼近

设 $\mathbb{k}$ 为赋范域， E, F 为 $\mathbb{k}$ 上的赋范空间， $f : E \rightarrow F$ ， $x \in E$ 。候选导数不是一个数，而是连续线性映射

$$A : E \longrightarrow_{L, \mathbb{k}} F.$$

记余项

$$r(h) = f(x + h) - f(x) - A(h).$$

称 A 是 f 在 x 的 Fréchet 导数，如果

$$\frac{\|r(h)\|}{\|h\|} \longrightarrow 0 \quad (h \rightarrow 0, h \neq 0). \quad (34)$$

等价地， $r(h) = o(\|h\|)$ 。这一定义同时包含两部分： A 必须线性且连续；扣除 $A(h)$ 后的误差必须比一阶增量更小。

HasFDerivAt 的类型

在 Mathlib 中，

$$\text{HasFDerivAt}(f, A, x) : \text{Prop},$$

其对象类型为

$$f : E \rightarrow F, \quad A : E \longrightarrow_{L, \mathbb{k}} F, \quad x : E.$$

Lean 写作 `HasFDerivAt f A x`。字母 F 表示 Fréchet，不是本讲前文的滤子变量。更底层的 `HasFDerivAtFilter` 确实允许把局部滤子作为参数。

式 (34) 仍然是极限陈述，可写成

$$\text{Tendsto} \left(h \mapsto \frac{\|f(x + h) - f(x) - A(h)\|}{\|h\|} \right) \mathcal{N}[\neq](0) \mathcal{N}(0).$$

Mathlib 的实际定义使用 little-o，使零增量与一般标量域的技术细节更自然，并能直接复用渐近分析 API。

例 4.1：连续线性映射的导数就是自身

若 $A : E \longrightarrow_{L, \mathbb{k}} F$ ，则

$$A(x + h) - A(x) - A(h) = 0.$$

余项恒为零，所以 A 在每一点的 Fréchet 导数都是 A 。这也是“线性化”一词的基准：非线性函数在局部由一个线性映射逼近。

4.2 为什么以 Fréchet 导数为基础

若从一元差商

$$\frac{f(x + h) - f(x)}{h}$$

出发，定义依赖定义域和值域都是标量域；到了 $\mathbb{R}^n \rightarrow \mathbb{R}^m$ ，就必须另行引入 Jacobian；到了函数空间，还要再次解释无穷维导数。Fréchet 定义一次覆盖这些情形。

更重要的是，导数的复合规律在连续线性映射层恰好就是函数复合：若

$$Df(x) : E \rightarrow_{L, \mathbb{K}} F, \quad Dg(f(x)) : F \rightarrow_{L, \mathbb{K}} G,$$

则复合映射的候选导数有唯一自然类型

$$Dg(f(x)) \circ Df(x) : E \rightarrow_{L, \mathbb{K}} G.$$

坐标矩阵、方向导数与一元导数都是这一映射在具体基或具体方向上的表示。

结构选择

Fréchet 可微强于仅有所有方向导数：后者不自动保证各方向近似能组成一个连续线性映射，也不自动给出统一的一阶余项控制。Mathlib 选择 Fréchet 导数，是因为它具有稳定的复合、乘积与隐函数理论，而不是因为差商无法形式化。

4.3 HasDerivAt：一元导数的接口

当定义域和值域都是标量域 $\mathbb{K}$ 时，每个连续线性自映射都由“乘以某个标量”给出。因此

$$\text{HasDerivAt}(f, f', x)$$

表示 f 在 x 的 Fréchet 导数是 $h \mapsto f'h$ 。类型为

$$f : \mathbb{K} \rightarrow \mathbb{K}, \quad f' : \mathbb{K}, \quad x : \mathbb{K}.$$

Lean 写作 `HasDerivAt f f' x`。它保留候选导数作为命题参数，比直接使用 `deriv f x` 更适合证明：定理的输出不仅给出一个值，还携带“这个值确实是导数”的证据。

两个导数接口

```
variable ℓ E F : Type* [NontriviallyNormedField ℓ]
variable [NormedAddCommGroup E] [NormedSpace ℓ E]
variable [NormedAddCommGroup F] [NormedSpace ℓ F]

#check ContinuousLinearMap
#check HasFDerivAt
#check HasFDerivAt.comp
#check HasDerivAt
#check HasDerivAt.comp
```

4.4 链式法则是连续线性映射的复合

设 $f : E \rightarrow F$, $g : F \rightarrow G$ ，并有

$$h_f : \text{HasFDerivAt}(f, A, x), \quad h_g : \text{HasFDerivAt}(g, B, f(x)).$$

Mathlib 的结构化链式法则给出

$$h_g.\text{comp}(x, h_f) : \text{HasFDerivAt}(g \circ f, B \circ A, x). \quad (35)$$

类型保证 $B \circ A$ 的方向正确；不匹配的线性映射无法组成这个项。

一元情形中，连续线性映射复合退化为标量乘法。若

$$h_f : \text{HasDerivAt}(f, f', x), \quad h_g : \text{HasDerivAt}(g, g', f(x)),$$

则

$$h_{g \circ f} : \text{HasDerivAt}(g \circ f, g' f', x). \quad (36)$$

这正是

$$(g \circ f)'(x) = g'(f(x))f'(x),$$

但 Lean 版本把函数、点、两个局部可微证据和导数复合全部保留在类型中。

链式法则的结构化陈述

```
variable f g : ℝ → ℝ f' g' x : ℝ
```

```
example (hf : HasDerivAt f f' x)
  (hg : HasDerivAt g g' (f x)) :
  HasDerivAt (g ∘ f) (g' * f') x :=
  hg.comp x hf
```

4.5 常见函数求导：由原子定理组合

Mathlib 为恒等、常值、加法、乘法、幂、倒数、指数、对数与三角函数提供局部导数定理。典型接口包括

$$\begin{aligned} \text{id}'(x) &= 1, \\ (f + g)'(x) &= f'(x) + g'(x), \\ (fg)'(x) &= f'(x)g(x) + f(x)g'(x), \\ (x^n)' &= nx^{n-1}, \\ (\exp)'(x) &= \exp x, \\ (\sin)'(x) &= \cos x, \\ (\cos)'(x) &= -\sin x. \end{aligned}$$

每条等式在 Lean 中首先以 HasDerivAt 命题出现，再可通过方法.deriv 取出 deriv 的值。

例 4.2: $\sin(x^2)$ 的导数

内层 $f(x) = x^2$ 在 x 的导数为 $2x$ ，外层 $g(y) = \sin y$ 在 x^2 的导数为 $\cos(x^2)$ 。由式 (36)，

$$\frac{d}{dx} \sin(x^2) = \cos(x^2) \cdot 2x.$$

Lean 证明的核心不是重新展开差商，而是把 hasDerivAt_pow 与 Real.hasDerivAt_sin 交给 HasDerivAt.comp。这正是结构化定理的可组合性。

例 4.3: 多变量复合不需要 Jacobian 坐标

设 $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$ 、 $g : \mathbb{R}^n \rightarrow \mathbb{R}^k$ 。一旦得到连续线性映射 $A = Df(x)$ 与 $B = Dg(f(x))$ ，链式法则的结论就是 $B \circ A$ 。若随后选定标准基，它的矩阵才是 Jacobian 矩阵乘积 $J_g(f(x))J_f(x)$ 。

Mathlib 把无坐标对象作为基础，矩阵表示留给真正需要坐标计算的场合。

与前三章的联系

Fréchet 导数定义中的 little-o 由 Filter 参数化；可微蕴含连续，则通过余项极限推出；链式法则把两个局部线性近似组合。微分理论没有离开 Filter，而是在 Filter 极限之上增加了赋范线性结构。

5 第五章 积分、收敛与分析形式化的特点

积分把分析的抽象从局部结构推向全局结构。拓扑决定“靠近”，测度决定“大小可忽略”；Lebesgue 积分正是围绕可测性、零测集与极限定理组织的积分理论。

5.1 Measure: 集合大小的类型化对象

在类型 α 上先给定可测空间结构 `[MeasurableSpace  $\alpha$ ]`，指定哪些集合允许测量。一个测度

$$\mu : \text{Measure}(\alpha)$$

把可测集送到扩充非负实数

$$\mu(S) \in \mathbb{R}_{\geq 0\infty},$$

满足 $\mu(\emptyset) = 0$ 与可列可加性。值域必须包含 $+\infty$ ，因为整个空间或某些集合可以具有无穷测度；这正是使用 `ENNReal` 作为值域的原因。

Measure 的类型

$$[\text{MeasurableSpace } \alpha], \quad \mu : \text{Measure}(\alpha), \quad \mu : \text{Set}(\alpha) \rightarrow \mathbb{R}_{\geq 0\infty}.$$

实际使用时，可测结构由 `typeclass` 参数提供。测度是定义在同一底层类型上的额外结构；Lebesgue 测度、计数测度与 Dirac 测度可以共享可测集合，却给出不同大小。

例 5.1: 三种测度回答不同问题

在 $\mathbb{R}$ 上，Lebesgue 测度给区间 $[a, b]$ 大小 $b - a$ ；计数测度给有限集其元素个数，给无限集 $+\infty$ ；Dirac 测度 δ_x 只检测集合是否含 x 。同一个函数相对于不同测度可以有不同的可积性和积分值，所以正式陈述必须把 μ 作为参数或 `typeclass` 中的默认测度。

测度的入口

```
open MeasureTheory
variable  $\alpha : \text{Type}^*$  [MeasurableSpace  $\alpha$ ]

#check Measure          -- Type  $u \rightarrow \text{Type } u$ 
#check (MeasureTheory.Measure.dirac)
#check Measure.count
#check volume
```

5.2 为什么以 Lebesgue 积分而非 Riemann 积分为基础

Riemann 积分从区间分割与和式逼近出发，适合初等计算；Lebesgue 积分按函数值层次与测度组织，适合极限过程。Mathlib 的通用积分基础采用 Lebesgue/Bochner 框架，主要原因是：

1. 定义域不必是实区间，可以是任意可测空间；
2. 改变零测集上的函数值不改变积分；
3. 单调收敛、Fatou 引理与控制收敛定理能在自然假设下陈述；
4. 可以统一概率空间、离散求和、实积分与 Banach 空间值积分。

这并不否定 Riemann 积分。连续函数在紧区间上的两种积分一致，区间积分记号仍然适合微积分计算；但当积分需要与函数列极限交换时，Lebesgue 理论提供更稳定的基础接口。

5.3 MeasureTheory.integral 与 Integrable

对 Banach 空间值函数 $f : \alpha \rightarrow E$ ，Mathlib 的 Bochner 积分写作

$$\int x, f(x) d\mu,$$

Lean 记号是 $\int x, f x \partial \mu$ ，底层函数为 `MeasureTheory.integral`。源码模块位于 `Mathlib.MeasureTheory.Integral` 层级；这里 `Integral` 是模块族名称，实际常用定义仍在 `MeasureTheory` 命名空间中。

`Integrable f μ` 不只是“积分符号能写出来”。其数学内容包括：

1. f 几乎处处强可测；
2. 范数具有有限积分，即 $\int \|f(x)\| d\mu < \infty$ 。

对实值函数，这就是绝对可积性。Mathlib 把积分定义成全函数；在非可积的情形采用约定值，因此需要积分定理时通常应显式携带 `Integrable` 假设，不能把“表达式有类型”误当成“数学积分存在”。

例 5.2：有限区间上的连续函数

若 $f : \mathbb{R} \rightarrow \mathbb{R}$ 在 $[a, b]$ 连续，则它在该区间相对于 Lebesgue 测度可积。区间积分

$$\int_a^b f(x) dx$$

由测度积分构造，并处理 $a > b$ 时的定向符号。初等微积分的基本定理建立在这一通用积分之上，而不是另设一套互不相干的 Riemann 基础。

积分与可积性的类型

```
open MeasureTheory
variable α E : Type* [MeasurableSpace α]
variable [NormedAddCommGroup E] [NormedSpace ℝ E] [CompleteSpace E]
variable (f : α → E) (μ : Measure α)

#check MeasureTheory.integral
#check Integrable
#check AESTronglyMeasurable
#check HasFiniteIntegral
```

5.4 几乎处处： $\forall^\mu$ 是一个 Filter 量词

给定测度 μ ，Mathlib 定义滤子

$$\text{ae}(\mu) : \text{Filter}(\alpha),$$

其中

$$S \in \text{ae}(\mu) \iff \mu(S^c) = 0. \quad (37)$$

于是“ P 几乎处处成立”就是普通的 `Eventually`：

$$\forall^\mu x, P(x) \iff \forall^f x \text{ in } \text{ae}(\mu), P(x). \quad (38)$$

Lean 记作 $\forall^m x, P x$; 若使用默认测度, 也常写 $\forall^m x, P x$ 。

例 5.3: 零测集上的修改

有理数集 $\mathbb{Q} \subseteq \mathbb{R}$ 的 Lebesgue 测度为零。函数

$$f(x) = \mathbf{1}_{\mathbb{Q}}(x)$$

几乎处处等于零, 记作 $f =^f [\text{ae}(\mu)] 0$, 所以其 Lebesgue 积分为零。它在每一点都不连续, 说明“逐点拓扑性质”与“几乎处处测度性质”是两种不同结构; Filter 让二者共享 eventual equality 的推理接口, 却不会混淆各自含义。

例 5.4: 几乎处处不等于处处

若 $f = g$ 几乎处处, 则它们的积分相同, 但不能推出 $f(x) = g(x)$ 对每个 x 成立。反之, 逐点相等当然推出几乎处处相等。形式化中从强命题降到弱命题通常由 `Filter.Eventually.of_forall` 完成; 从几乎处处结论恢复逐点结论则一般不合法。

5.5 控制收敛定理如何表达

设 $f_n : \alpha \rightarrow E$, $f : \alpha \rightarrow E$, $g : \alpha \rightarrow \mathbb{R}$ 。控制收敛定理的一组典型假设为:

1. f_n 具有适当的可测性;
2. 几乎处处逐点收敛:

$$\forall^\mu x, \text{Tendsto}(n \mapsto f_n(x)) \text{ atTop } \mathcal{N}(f(x)); \quad (39)$$

3. 存在可积控制函数 g , 使每个 n 都有

$$\forall^\mu x, \|f_n(x)\| \leq g(x). \quad (40)$$

结论包括 f 可积, 并且

$$\text{Tendsto} \left(n \mapsto \int x, f_n(x) \, d\mu \right) \text{ atTop } \left(\mathcal{N} \left(\int x, f(x) \, d\mu \right) \right). \quad (41)$$

式 (39) 中嵌套了两种 Filter: 外层 $\text{ae}(\mu)$ 忽略零测集, 内层 atTop 忽略数列的有限开头。式 (41) 又使用 atTop 与积分值的邻域滤子。这不是额外复杂化, 而是把“在哪个变量上允许忽略什么”逐层写清。

控制收敛相关接口

`open Filter MeasureTheory`

`#check MeasureTheory.ae`

`#check MeasureTheory.ae_iff`

`#check MeasureTheory.tendsto_integral_filter_of_dominated_convergence`

`#check MeasureTheory.hasFiniteIntegral_of_dominated_convergence`

例 5.5：为什么逐点收敛本身不够

在 $(0, 1)$ 上令 $f_n = n\mathbf{1}_{(0, 1/n)}$ 。对每个固定 $x > 0$ ，最终有 $f_n(x) = 0$ ，所以 $f_n(x) \rightarrow 0$ ；但

$$\int_0^1 f_n(x) dx = 1$$

不趋于 0。这里不存在一个共同的可积控制函数。形式化定理把可测性、几乎处处收敛与统一控制分别作为假设，防止把不充分的点态信息误用为积分收敛。

5.6 全篇收束：分析形式化的三个特点**5.6.1 高度抽象，但抽象有明确职责**

Filter 管理趋近与最终性，Topology 管理开集和连续，Measure 管理零测集与可积性。这些结构不是同义包装：每一层都规定可用的操作和定理。抽象的收益是让结论在正确的最弱接口上成立，并能跨越实数、函数空间、扩充数系与概率空间复用。

5.6.2 大量 typeclass 用于传递结构

一个分析定理的上下文常包含

[TopologicalSpace X] （或由 [MetricSpace X] 诱导），
[NormedAddCommGroup E], [MeasurableSpace α].

typeclass 让 Mathlib 自动寻找由范数诱导的度量、由度量诱导的拓扑，以及标准类型上的默认实例。使用者无需反复传参，但必须理解每个实例提供什么；若同一类型上需要两个不同拓扑或测度，就应显式区分，不能依赖默认推断。

5.6.3 存在性与条件管理比代数更复杂

代数恒等式常在给定运算结构后处处成立；分析命题则频繁依赖非退化、完备、可测、可积、几乎处处与支配条件。形式化会迫使这些条件进入定理陈述：

$F.\text{NeBot}$, $\text{CompleteSpace}(E)$, $\text{Integrable}(f, \mu)$, $\forall^\mu x, P(x)$.

这不是形式系统制造的负担，而是分析中存在性问题本来就需要的边界。证明的主要工作往往不是化简表达式，而是把条件放到正确的局部区域，并证明它们在复合、极限或积分下得到保存。

最终视角

本讲的抽象路径可以压缩为三句话：

1. 用结构说明哪些观察是合法的：距离、邻域、可测集；
2. 用 Filter 说明哪些例外可以忽略：远离局部的点、有限开头、零测集；
3. 用类型化定理说明结构如何组合： Tendsto.comp 、 HasFDDerivAt.comp 、控制收敛。

理解这些接口后，Mathlib 中的分析不再是一组特殊记号，而是一套围绕局部性、收敛与条件管理组织起来的统一语言。